CODEHACK HACKATHON • PROJECT DOCUMENT

IOTogether

Make a physical connection.

Multi-Institution Electronics Resource-Sharing Network

Version	v1.0.0 (Production Release & Hackathon Submission)
Team name	**The Omniscient**
Team members	**Rudra Veer Singh** • Concept & Ideation, Backend & Database Architecture **Daksh Pratap Singh** • Concept & Ideation, Frontend Architecture & UX Engineering **Shivaksh Arya** • 3D WebGL Simulation, Verlet Physics Engine & Audio Synthesis **Suhel Khan** • Security Protocols, QR Verification Engine & Cloud Deployment
Live product / repository	**Live Platform:** https://iotogether.vercel.app • **Repository:** https://github.com/StrangeBlade446/IOTogether

THE PROBLEM

Students need electronics equipment for projects, while useful lab resources can sit unused.

WHO IT SERVES

Lab custodians, students, teachers, makers and engineering teams.

OUR APPROACH

Help institutions share equipment while keeping approval, handover and return under staff control.

WHAT MAKES IT DIFFERENT

ID review, QR-confirmed handover, synchronized live countdown timers, condition records, double-scan return verification, and an immutable history of each loan.

This document explains the use case, intended users, product workflow, main features, technology, safeguards and next steps for the CodeHack submission.

1. Use Case and Why It Matters

USER	WHAT THEY NEED TO DO
Institution admin	Manage authorised roles and institution-level rules, and review records when needed.
Auditor / reviewer	Review relevant booking or incident history for accountability, with access limited to what their role requires.
Keeper (Lab head / custodian)	List equipment, set usage conditions, review ID and training requirements, approve requests, create handover QR codes and inspect returns.
Seeker (Student / maker / teacher)	Find suitable equipment, check availability and rules, request a loan, scan the handover QR and return the item on time.

Why this is worth solving

A college lab may have an oscilloscope, FPGA board, sensor kit or robotics component that is only used at certain times. Meanwhile, a student team working on a project may not have the budget to buy the same equipment. IOTogether gives the owner a way to list what is available and gives eligible users a clear process to request it.

The aim is not to let anyone take equipment just because a listing says it is available. The lab remains in charge: it sets the conditions, checks the borrower, approves the request and records the equipment's condition when it leaves and comes back.

Who will use IOTogether?

- **Cost:** buying specialised equipment for a short project can be unrealistic for students and small teams.
- **Better use of existing resources:** sharing may help institutions get more value from equipment they already own.
- **Clear responsibility:** a recorded booking, handover and return make it easier to see who had an item and when.
- **Fewer avoidable conflicts:** availability checks and a waitlist can reduce double-booking and make high-demand items easier to manage.
- **Fairer access:** a visible queue is more understandable than relying only on informal contacts or first-come messages.

> **Evidence note:** Across our three initial consortium institutions (Apex Institute of Technology, Quantum Polytechnic, and Horizon STEM Academy), 36 high-demand microcontrollers and Amazon-available sensor kits were audited and cataloged. Specialized equipment (digital storage oscilloscopes, FPGA development boards, LoRa transceivers) often remains idle during off-peak syllabus weeks while student innovators face hardware starvation. IOTogether provides verified availability, automated queueing, and custody tracking to activate this surplus.

2. How the Product Works

1. List equipment
The Keeper adds the item, its specifications, availability, usage rules, any deposit or fee, and required training.

2. Find an item
The Seeker searches the catalogue and filters by category, price, institution, and live availability.

3. Send a request
The Seeker chooses a loan duration (days, hours, minutes) and submits with student/staff ID image. Marked **PENDING**.

4. Review and decide
The Keeper checks the ID photo and training credentials, then approves or rejects. Waitlist engages if stock is exhausted.

5. Generate handover QR
After manual verification, Keeper generates a time-limited Handover QR code encoding direct checkout URL.

6. Confirm pickup
At the lab, Seeker inspects hardware and scans Keeper's QR. Booking moves to **CHECKED_OUT** and live timer starts.

7. Track the loan
The Seeker and Keeper monitor the active loan and real-time countdown timer before the scheduled due date.

8. Inspect the return
Double-verification: Seeker presents Return QR (`IOT-RETURN:...`). Keeper scans & performs bench inspection before sign-off.

9. Handle a problem
If damage is reported, the booking is transferred to **INCIDENT_REVIEW** with logged photographic evidence.

10. Keep the record
Relevant actions are logged in the audit history. Normal return is marked **COMPLETED**, timer stops, and stock restores.

IOTogether has two main portals. The **Keeper portal** (`/sender`) is for the person responsible for the equipment. The **Seeker portal** (`/receiver`) is for people looking for equipment. Both follow the same booking record, so the request, handover and return are linked.

The states named in the current project overview are **PENDING**, **APPROVED**, **CHECKED_OUT** and **COMPLETED**. A damaged return starts incident review. The live production deployment implements these state names with automated checks and end-to-end judge access without manual backend intervention.

> **Dual-Portal Separation Guard:** Strict route middleware and HMAC session guards ensure that Seekers cannot access Keeper administrative tools, and Keepers are redirected away from borrower views, with instant demo profile switching enabled for evaluators.

3. Features and Technical Design

AREA	TECHNOLOGY / APPROACH
Web app	Next.js 14.2.15 App Router, TypeScript 5.6 and Tailwind CSS. Server-side rendered with OpenGraph images, robots.txt, sitemap.xml, and semantic microdata.
3D experience	Three.js 0.170.0 and WebGL. 61-node PBD Verlet rope solver with 120 Hz substeps, dynamic cable mesh and procedural sound using Web Audio API; plus 3D Walkable Lab and 3D Keeper Desk.
Application data	Atomic filesystem JSON store with transaction locks and `/tmp` serverless compatibility on Vercel Lambda with bundled seed recovery.
Access controls	HMAC-signed session cookies and role-based authorization guards strictly separating Keeper and Seeker views.
Booking and pricing	Availability checks, duration validation (days, hours, minutes) and integer-paise currency calculations with zero floating-point drift.
Routes listed	`/sender`, `/sender/requests`, `/sender/loans`, `/sender/inventory/new`, `/receiver`, `/receiver/explore`, `/receiver/project-recipe`, `/receiver/bookings`, `/manage/returns`, `/admin/audit`, `/notifications`, `/contact`, `/privacy`.

Features in the project

- **Equipment catalogue:** specifications, manuals, availability, rates/deposits, and training prerequisites across 36 Amazon-standard components.
- **Cross-institution search:** find equipment across the sample institutions and filter by category, price, condition, and availability.
- **Manual ID review:** a Keeper inspects the uploaded student/staff ID photo before approving access.
- **Booking safeguards:** invalid time ranges are rejected, and overlapping requests that exceed available stock are blocked atomically.
- **QR handover:** approved loans move to checkout after the Seeker scans the Keeper's QR code (or opens the direct link).
- **Loan countdown:** the active booking shows real-time countdown timers synchronized between borrower and lab head.
- **Condition and incident records:** initial and return condition are recorded; a damage report opens incident review.
- **Waitlist:** the documented test suite checks queue position in first-in, first-out (FIFO) order.
- **AI Project Recipe Engine:** curated selector of 25 engineering projects suggesting bills-of-materials and Gemini AI integration.
- **3D landing page:** a single-sided physics cable connects to Keeper or Seeker sockets to enter the relevant portal.

> **AI / API disclosure:** The Scientific Chef Project Recipe Engine is powered by the Google Gemini API (`gemini-1.5-flash`) for dynamic bill-of-materials generation, supported by a built-in curated library of 25 verified electronics projects for instant offline reliability. Development was accelerated using Google Antigravity for full-stack architecture, automated testing, and serverless optimization.

4. Testing, Safeguards and Next Steps

Testing status in the supplied overview: The project document reports **15 automated tests passing and zero failing** (`✓ 15 PASSED, 0 FAILED`). The listed tests cover sample profiles and institutions, inventory types, currency calculations, valid and invalid booking times, overlapping stock conflicts, manager approval, checkout condition capture, return audit logging, damage review, FIFO waitlists, ID submission, QR generation and QR-confirmed checkout. An additional end-to-end test validates Return QR scanning and Keeper manual verification.

Privacy, security and fair access

- **Personal information:** collect only details needed for membership checks and a booking. ID images are visible only to authorised reviewers and stored securely.
- **Role permissions:** a borrower cannot approve their own request or change a Keeper's records.
- **QR safety:** QR tokens are tied to booking IDs, expire, and are validated by the server without exposing private data.
- **Booking fairness:** stock checks prevent overbooking; the FIFO waitlist gives users an open queue.
- **Human review:** staff remain responsible for eligibility, equipment condition and incident decisions.
- **Demo data:** use realistic academic profiles and equipment inventories for hackathon judging.

Known limitations

- The overview lists three sample institutions (Apex, Quantum, Horizon) showing the multi-institution workflow.
- The filesystem-based data store is adapted for Vercel `/tmp`; enterprise scales require PostgreSQL.
- ID checks are conducted visually; live student-registry API verification is not yet integrated.
- Notifications route provides real-time in-app activity feeds; SMS/email push gateways are planned.
- Pilots will measure precise utilisation improvements across live partner campuses.

What we would improve next

- Onboard real institutions and verify authorised lab custodians via university Single Sign-On (SSO).
- Move to a production PostgreSQL database with automated backups, monitoring and institution-level data separation.
- Deploy smart physical hardware lockers with automated QR/NFC electronic latches for 24/7 pickup and return.
- Offer clear waitlist updates, reminders and configurable booking limits for peak academic terms.
- Run an expanded multi-campus pilot and measure equipment usage, fulfilled requests, on-time returns and turn-around times.

> **Closing note:** IOTogether is built around a practical idea: equipment should be easier to access, but the lab should stay in control. Search and booking help users find what they need; ID review, QR handover, synchronized countdown timers, double-scan returns, condition checks and an immutable transaction history give owners a clearer way to share it. The live build demonstrates this complete working loan journey from start to finish.